



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Летучка № 9
по курсу «Языки и методы программирования»

«Сокеты в C++. TCP-сервер и клиент»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Реализация класса Socket	1
Заголовочный файл socket.hpp	2
Конструктор	2
Серверная часть	2
bind_and_listen()	2
wait_for_message()	3
Клиентская часть	3
connect_to_server()	3
send_message()	4
main()	4
Сервер	4
Клиент	4
Компиляция и запуск	5
Makefile	5
Запуск	5
Ключевые понятия	5
Вывод	6

Цель работы

Реализовать TCP-сервер и клиент на C++ с использованием сокетов. Сервер принимает сообщения от клиентов и отправляет их обратно (режим ECHO).

Задачи:

- Изучить API сокетов в C++.
- Реализовать класс Socket для работы с TCP.
- Создать сервер с функцией accept и recv/send.
- Создать клиент с функцией connect.

Теоретические сведения

Сокеты — это программный интерфейс для межпроцессного взаимодействия.

Основные функции:

- socket() — создание сокета
- bind() — привязка к адресу и порту
- listen() — прослушивание порта
- accept() — принятие соединения
- connect() — подключение к серверу
- send() — отправка данных
- recv() — получение данных

Реализация класса Socket

Класс Socket инкапсулирует работу с TCP-сокетами.

Заголовочный файл socket.hpp

socket.hpp

```
#include <arpa/inet.h>
#include <cstring>
#include <iostream>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>

struct Socket {
    int serverSocket;
    sockaddr_in serverAddress;
    int port;
    const char *target_ip;

    Socket(int port, const char *target_ip) : port(port),
    target_ip(target_ip) {
        serverSocket = socket(AF_INET, SOCK_STREAM, 0);

        serverAddress.sin_family = AF_INET;
        serverAddress.sin_port = htons(port);

        inet_pton(AF_INET, target_ip, &serverAddress.sin_addr);
    }

    ~Socket() { close(serverSocket); }
};
```

Конструктор

В конструкторе создается сокет и инициализируется адрес:

- `socket(AF_INET, SOCK_STREAM, 0)` — создание TCP-сокета
- `htons(port)` — преобразование порта в сетевой порядок байтов
- `inet_pton()` — преобразование IP-адреса из текстового в бинарный формат

Серверная часть

`bind_and_listen()`

socket.hpp

```
void bind_and_listen() {
    bind(serverSocket, (struct sockaddr *)&serverAddress,
        sizeof(serverAddress));
    listen(serverSocket, 5);
}
```

- `bind()` — привязка сокета к адресу и порту
- `listen()` — перевод сокета в режим прослушивания (очередь 5 клиентов)

`wait_for_message()`

socket.hpp

```
void wait_for_message() {
    sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);

    int clientSocket =
        accept(serverSocket, (struct sockaddr *)&client_addr,
        &addr_len);

    char client_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &client_addr.sin_addr, client_ip,
    INET_ADDRSTRLEN);

    char buffer[1024];
    while (true) {
        memset(buffer, 0, 1024);
        int bytesRead = recv(clientSocket, buffer, sizeof(buffer), 0);

        std::cout << "Received: " << buffer << std::endl;
        std::cout << "Sended back: " << buffer << std::endl;

        // ECHO: отправляем обратно
        send(clientSocket, buffer, bytesRead, 0);
    }
    close(clientSocket);
}
```

- `accept()` — принятие соединения от клиента
- `recv()` — получение данных от клиента
- `send()` — отправка данных клиенту (режим ECHO)
- `inet_ntop()` — преобразование IP-адреса в текстовый формат

Клиентская часть

`connect_to_server()`

socket.hpp

```
void connect_to_server() {
    connect(serverSocket, (struct sockaddr *)&serverAddress,
    sizeof(serverAddress));
}
```

send_message()

socket.hpp

```
void send_message(const char *message) {
    send(serverSocket, message, strlen(message), 0);
    char buffer[1024] = {0};
    int bytesRead = recv(serverSocket, buffer, sizeof(buffer), 0);
    if (bytesRead > 0) {
        std::cout << "Received from server: " << buffer << std::endl;
    }
}
```

main()

Сервер

server.cpp

```
int main() {
    Socket s(3896, "185.104.251.226");

    s.bind_and_listen();
    while (true) {
        s.wait_for_message();
    }
}
```

Клиент

client.cpp

```
int main() {
    Socket s(3896, "185.104.251.226");

    s.connect_to_server();

    std::string line;
    while (std::getline(std::cin, line)) {
        s.send_message(line.c_str());
    }
}
```

Компиляция и запуск

Makefile

Makefile

```
.PHONY: all run clean

all:
    g++ -o build/server server.cpp
    g++ -o build/client client.cpp

run_server: all
    ./build/server

run_client: all
    ./build/client

clean:
    rm -rf build
```

Запуск

server@host

```
$ make run_server &
```

client@local

```
$ make run_client
Hello, server!
Received from server: Hello, server!
```

Сервер получает сообщение и отправляет его обратно (режим ECHO).

Ключевые понятия

1. `socket()` — создание сокета
2. `bind()` — привязка к порту
3. `listen()` — прослушивание
4. `accept()` — принятие соединения
5. `connect()` — подключение к серверу
6. `send()/recv()` — отправка и получение данных
7. `htons()/inet_pton()` — преобразование адресов

Вывод

В ходе лабораторной работы были реализованы:

1. Класс Socket для работы с TCP-сокетами
2. TCP-сервер, принимающий соединения и отправляющий данные обратно (ECHO)
3. TCP-клиент для отправки сообщений серверу

Использованы системные вызовы socket API в C++ для сетевого взаимодействия.